Autonomy II Midterm Project Report

J. Poirier, S. Wiederman

March 15, 2026

Abstract

This report details the development and implementation of a ROS2-based control architecture for the Wlkata Mirobot manipulator, with the ultimate goal of a pure pursuit trajectory tracking system. The project incrementally constructs the robotic control pipeline, beginning with establishing serial communication and joint angle command nodes. Utilization of GUIs is then implemented, followed by optimization-based inverse kinematics solvers. Finally, a pure pursuit algorithm is implemented, enabling the robotic arm to autonomously follow user-drawn paths. RViz2 visualization is used throughout in conjunction. The successful integration of these components demonstrates the practical application of kinematic solvers and path-following algorithms in robotic systems.

Contents

1 Step 1: Preliminaries and Goals **4**

2 Step 2: Controlling Wlkata Mirobot **4**
- 2.1 Mirobot Commander Node . . . 4
- 2.2 Results . . . 5
- 2.3 Difficulties and Workarounds . . . 6

3 Step 3: GUI Bridge **6**
- 3.1 Writing the Node . . . 6
  - 3.1.1 Loading Robot Model . . . 6
  - 3.1.2 Setup Communication . . . 7
  - 3.1.3 Bridge The Message . . . 7
  - 3.1.4 Calculate Forward Kinematics . . . 7
  - 3.1.5 Extracting Constants . . . 8
- 3.2 Results . . . 8
- 3.3 Difficulties and Workarounds . . . 9

4 Step 4: The Inverse Kinematics Solver **9**
- 4.1 Writing Inverse Kinematics Node . . . 9
  - 4.1.1 Setup and Model Loading . . . 9
  - 4.1.2 The Objective Function . . . 10
  - 4.1.3 The Gradient Descent Optimization Loop . . . 11
- 4.2 Cartesian Sender Node . . . 12
  - 4.2.1 Writing the Cartesian Sender Node . . . 12
- 4.3 Results . . . 13
- 4.4 Difficulties and Workarounds . . . 14

5 Step 5: Pure Pursuit **14**
- 5.1 Node Architecture . . . 14
  - 5.1.1 Communication . . . 14
  - 5.1.2 Robot Model . . . 15
- 5.2 Phase 1 . . . 15
  - 5.2.1 Get Start Point . . . 15
  - 5.2.2 Publish Target Marker and Cartesian Command . . . 15
  - 5.2.3 Check Distance . . . 16
- 5.3 Phase 2 . . . 16
- 5.4 Launch File Update . . . 18
- 5.5 Results . . . 18
- 5.6 Difficulties and Workarounds . . . 18
  - 5.6.1 Cartesian Command Publisher . . . 18
  - 5.6.2 Look-Ahead Distance . . . 19

6 Conclusion **19**

7 Appendix **20**
7.1 Code 20
7.1.1 Robot Description Launch File 20
7.1.2 Mirobot Commander Node 21
7.1.3 GUI Bridge Node 22
7.1.4 Inverse Kinematics Solver Node 24
7.1.5 Cartesian Sender Node 27
7.1.6 Mirobot Pure Pursuit Node 29

1 Step 1: Preliminaries and Goals

This project has several preliminary steps to ensure proper connectivity between the Wlkata Mirobot robotic arm and ROS2 through a serial connection. The steps taken to establish a connection are described extensively in the project proposal document, and those steps were followed carefully. The only complication through these steps arose when attempting to set up the provided "mirobot_connect" and "mirobot_disconnect" scripts. These scripts did not function as expected, but working around them was simple. Instead of using the scripts, the connection was simply established using the Windows PowerShell whenever necessary. Another point of care was properly organizing the file structure when copying the provided packages. Ensuring the proper file path structure is extremely important when working with ROS2, and this step took some finesse, but ultimately did not cause any major issues.

The goals of this project are to get the Mirobot robotic arm moving in desired ways using various methods through both forward and inverse kinematics. Ultimately, the goal is to implement a pure pursuit go-to-goal algorithm through incremental steps, building the framework for the control of the robot up to the pure pursuit algorithm itself.

2 Step 2: Controlling Wlkata Mirobot

The majority of this section of the project is a simple guideline to follow for the construction of the project workspace. The launch files, however, are created by the authors. The template for the launch file was provided with the project documents, and updated continually as the development of the project continued. The launch script can be found in **Appendix section 7.1.1**.

2.1 Mirobot Commander Node

The first step in creating the control architecture for the Mirobot, aside from understanding the scripts provided, was to create the robot arm commander node. The full script for this node can be found in **Appendix section 7.1.2**. The goal of this node is to send specific joint angles to the driver node. The first part to this is to create a topic for the commander to publish the joint angles to, and for the driver to listen to. This is accomplished via the below lines of code.

```
        self.joint_pub = self.create_publisher(Float64MultiArray, "/
    mirobot/command_joints", 10)

        self.get_logger().info("Sender Node Ready.")
```

Next, the node must properly publish information to the created topic. This is accomplished via the below lines of code.

```
        msg = Float64MultiArray()
        msg.data = joints
        self.joint_pub.publish(msg)
```

The final step for this node is to construct the matrices and timings for the desired movements of the robot. Importantly, the time delay between sending the previous position and the next position must be significant enough to allow for the robot to complete the motion. Additionally, the use of the timer.sleep function is generally poor practice in the use of robotics, as it does not allow for any information to be received or sent during that time, but because of the simple nature of this node and that it is not subscribed to any topics, we can guarantee that it is safe to use in this case. Below are the lines of code used to orchestrate the movement of the robot.

```
def main(args=None):
rclpy.init(args=args)
sender = MirobotSender()

# [J1, J2, J3, J4, J5, J6]
home_pose = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
pose_A = [0.5, -0.2, 0.2, 0.0, 0.0, 0.0]
pose_B = [-0.5, 0.3, 0.5, 0.0, 0.0, 0.0]

try:
    sender.send_joints(home_pose)
    time.sleep(2) #generally bad practice, but this node never listens
for anything, so it's safe
    sender.send_joints(pose_A)
    time.sleep(2)
    sender.send_joints(pose_B)
    time.sleep(2)
except KeyboardInterrupt:
    pass
finally:
    sender.destroy_node()
    rclpy.shutdown()
    pass
```

2.2 Results

The results for this step were successful. The robot was successfully controlled via the mirobot_commander node. The results are shown in a screenshot including the RViz visualization and the physical robot in **Figure 1**.

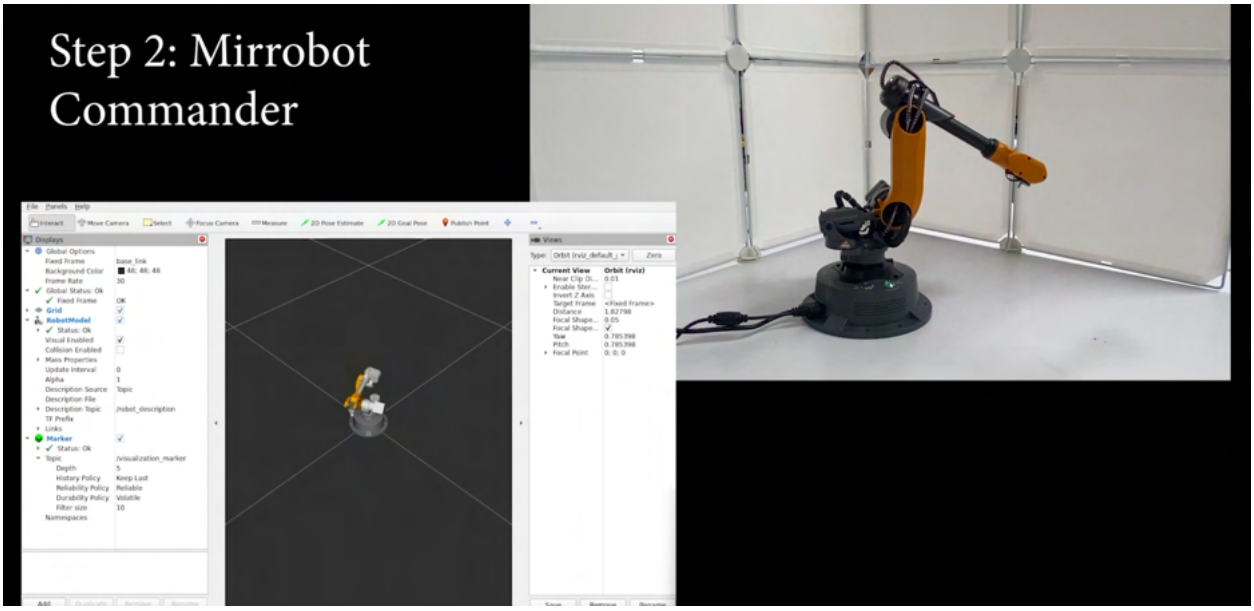


Figure 1: RViz2 and Physical Robot Results for Step 2.

2.3 Difficulties and Workarounds

This section did not provide many complications. The only issue that arose during the development in this section was ensuring proper connection to the physical robot arm. This turned out to be an issue with the WSL passthrough connection to the robot arm, and once that was resolved, the driver node, commander node, and RViz2 node all worked as expected.

3 Step 3: GUI Bridge

3.1 Writing the Node

The next section of this project was to create a node to bridge the RViz2 GUI with the mirobot driver node to move the physical robot using the GUI joint angle sliders. The full script of the GUI bridge can be found in **Appendix section 7.1.3**.

3.1.1 Loading Robot Model

To begin, the provided robot URDF file must be properly loaded. The below lines of code load the URDF file and properly initialize it to be used to gather information about the physical robot.

```
        urdf_file_name = 'mirobot_description.urdf'
        urdf = os.path.join(get_package_share_directory('
    mirobot_description'), 'urdf', urdf_file_name)
        # Load the chain using ikpy.chain.Chain.from_urdf_file()
        self.chain = ikpy.chain.Chain.from_urdf_file(urdf)
```

3.1.2 Setup Communication

Next, the topic communications must be set up. This node must subscribe to the topic "/gui/joint_state" to listen for the defined joint angles defined from the RViz2 GUI. This node must also publish to the topic "/mirobot/command_joints" to control the movements of the robot. These communications are established via the lines of code below.

```
        # TASK 2: SETUP COMMUNICATION
        # Create a Subscriber to Joint States from GUI (NOTE: You should
    subscribe to a different topic than the real robot to avoid conflicts,
    you can remap the topic when running joint state publisher gui)
        self.subscription = self.create_subscription(JointState, "/gui/
    joint_state", self.listener_callback, 10)

        # Create a Publisher to Robot Driver
        self.joint_pub = self.create_publisher(Float64MultiArray, "/
    mirobot/command_joints", 10)

        self.get_logger().info("GUI Bridge with FK Feedback Started.")
```

3.1.3 Bridge The Message

Now, with the topics properly set up within the node, code must be written to publish the appropriate information to the topic "/mirobot/command_joints". This is accomplished within the "listener_callback" function shown below. This code defines the message type and publishes the message.

```
        # TASK 3: BRIDGE THE MESSAGE
        # The GUI sends a JointState message (msg.position).
        # Create the new message, populate .data, and publish it.
        js_msg = Float64MultiArray()
        js_msg.data = msg.position
        self.joint_pub.publish(js_msg)
```

3.1.4 Calculate Forward Kinematics

This step calls the forward_kinematics() function from the ikpy.chain library. A base link at (0,0) must be appended to the forward kinematics calculation to properly calculate the positions and kinematics. The below line of code is used to calculate the forward kinematics of the robot.

```
        # TASK 4: CALCULATE FORWARD KINEMATICS
        # Use self.chain.forward_kinematics() to get the 4x4
    transformation matrix.
        # Note: You might need to prepend 0.0 to the joints list to
    account for the base link.
        transf_mat = self.chain.forward_kinematics([0.0] + list(msg.
    position))
```

3.1.5 Extracting Constants

The final step of this node is to extract the relevant constants. X, Y, and Z positions, as well as Roll, Pitch, and Yaw are required constants for the control of the robot. The lines of code below are used to extract these constants.

```
        # TASK 5: EXTRACT & PRINT
        # Extract X, Y, Z from the matrix (Index [0,3], [1,3], [2,3])
        X = float(transf_mat[0,3])
        Y = float(transf_mat[1,3])
        Z = float(transf_mat[2,3])
        # Use the helper function above to get rotation.
        rotations = self.rotation_matrix_to_euler(transf_mat[0:3, 0:3]) #
  list, [roll, pitch, yaw]
        # Log the result to the console.
        self.get_logger().info(f"Positions = [{X}, {Y}, {Z}]")
        self.get_logger().info(f"Rotations = {rotations}")
```

Additionally, the helper function "rotation_matrix_to_euler" is defined to assist in extracting the angles. This helper function was provided, and is shown in the full gui_bridge.py script.

3.2 Results

This step was again fully successful. The connection between the GUI sliders and RViz, as well as the GUI and the physical robot worked exactly as expected. The screenshot in **Figure 2** below shows the RViz visualization and the physical robot position from a randomized position given by the GUI.

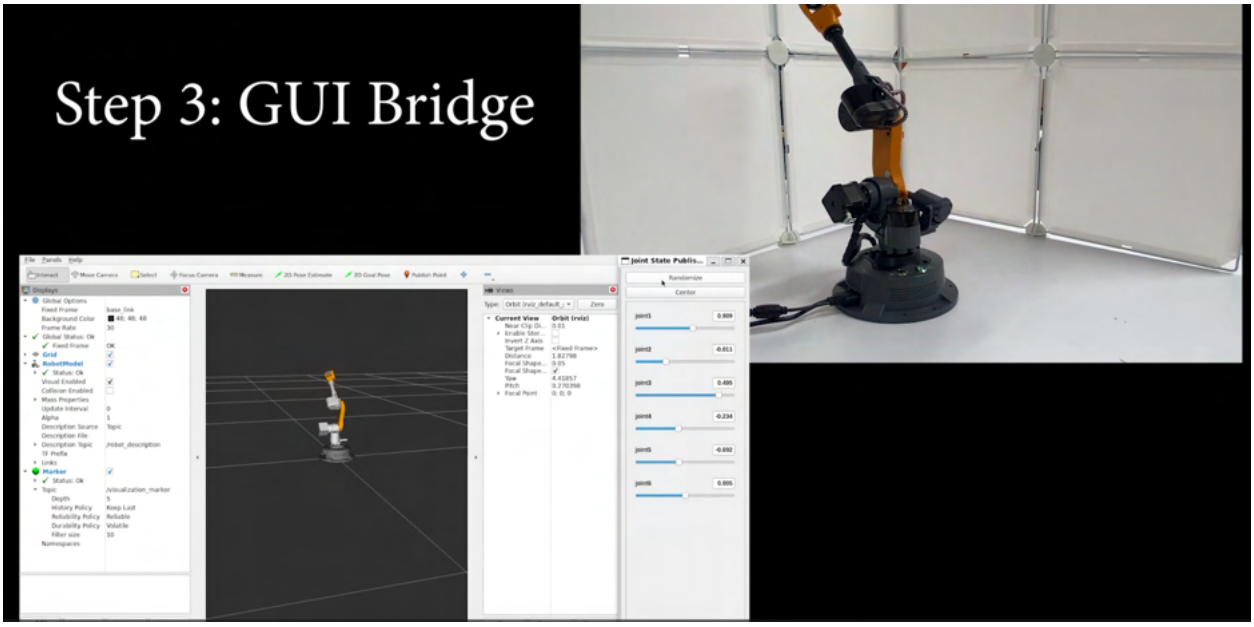


Figure 2: RViz2 and Physical Robot Results for Step 3.

3.3 Difficulties and Workarounds

This section also did not have any major complications during the development of the node. One major catch in this node is to properly label the topics, as any inconsistencies will result in miscommunications and will not allow the full pipeline to work properly. Furthermore, these problems will generally not show an error in the terminals, making them difficult to notice sometimes. This node functioned exactly as expected, allowing control of the physical robot through the RViz2 joint angle sliders GUI.

4 Step 4: The Inverse Kinematics Solver

4.1 Writing Inverse Kinematics Node

This node serves to solve inverse kinematics problems of the Wlkata Mirobot robotic arm so that the joint angles can be used to control the robot. The full script for this node is shown in **Appendix section 7.1.4**.

4.1.1 Setup and Model Loading

To start this node, the proper subscriptions and publications must be defined. This node subscribes to the topic "/mirobot/command_cartesian" to receive the positions from the cartesian_sender node (described later in this report), and publishes to the "/mirobot/-command_joints" topic to send joint angle information to the mirobot driver node. This communication setup is shown in the code below.

```
        # 1. SUBSCRIBE TO CARTESIAN TARGETS (POSITION + ORIENTATION)
        self.subscription = self.create_subscription(Float64MultiArray, "/
    mirobot/command_cartesian", self.cartesian_callback, 10)

        # 2. PUBLISH TO ROBOT DRIVER
        self.joint_pub = self.create_publisher(Float64MultiArray, "/
    mirobot/command_joints", 10)
```

Again, the URDF model must also be loaded. This is accomplished using the below lines of code, very similar to the previous GUI bridge node. Notably, in this code, it is specified that the base link (link 0) is set to false, meaning that it is not considered in the calculations of the inverse kinematics. It exists only as a requirement for the forward kinematics and RViz2 compatibility.

```
        urdf_file_name = 'mirobot_description.urdf'
        urdf = os.path.join(get_package_share_directory('
    mirobot_description'), 'urdf', urdf_file_name)

        # We have 7 links total (Base + 6 Motors).
        # We set index 0 (Base) to False, and indices 1-6 (Motors) to True
    .
        self.chain = ikpy.chain.Chain.from_urdf_file(
            urdf,
            active_links_mask=[False, True, True, True, True, True, True]
```

```
        )
```

Lastly, joint limits must also be defined for the use of inverse kinematics. These are physical limits that the robot cannot move into due to physical constraints, and defining them allows the inverse kinematics solver to know that solutions that fall outside of those limits are not valid. The below lines of code are used to extract and define the joint limits.

```
        # 5. EXTRACT JOINT LIMITS (BOUNDS)
        self.joint_limits = []
        for i in self.motor_indices:
            self.joint_limits.append(self.chain.links[i].bounds)

        # Seed: Where the robot starts looking for a solution (Start at "
    all zeros")
        self.current_full_joints = np.zeros(len(self.chain.links))

        # Give the shoulder and elbow a small "human-like" bend (in
    radians)
        # This helps prevent singularities on the very first move.
        self.current_full_joints[2] = 0.1  # Joint 2
        self.current_full_joints[3] = 0.2  # Joint 3

        self.get_logger().info("Gradient IK Solver Ready.")
```

4.1.2 The Objective Function

The objective of this section of code is to calculate an objective error to be used for the gradient descent optimization. First, the list of joint angles must be formatted properly to match the URDF file. This is done using the below lines of code.

```
        full_joints = self.current_full_joints.copy() #avoid overwirting
    the original
        for i, angle in enumerate(active_joints):
            full_joints[self.motor_indices[i]] = angle
```

Next is to run forward kinematics and extract the calculated positions and rotations. This is accomplished using the below lines of code.

```
        transf_mat = self.chain.forward_kinematics(full_joints)

        X = float(transf_mat[0,3])
        Y = float(transf_mat[1,3])
        Z = float(transf_mat[2,3])
        # Use the helper function above to get rotation.
        cur_rotation = transf_mat[0:3, 0:3] #list, [roll, pitch, yaw]
```

Finally, the error from the goal position and the calculated position must be found. This is done by using the norm between the target position and the calculated position, along with a heuristic weighting. This is shown in the code below.

```
        pos_error = np.linalg.norm(self.target_position - transf_mat[0:3,
    3])
        rot_error = np.linalg.norm(self.target_rotation - cur_rotation)

        # TASK 4: RETURN WEIGHTED COST
        # Return the total error. (Hint: Position error is usually
    weighted higher than rotation).
        return (pos_error * 100.0) + (rot_error * 2.0)
```

4.1.3 The Gradient Descent Optimization Loop

This optimization loop is handled in the "cartesian_callback" function that is defined in the node script. First, the inputs to the optimization must be parsed and converted to the correct units, the target position and rotations must be setup, and the last known position must be saved for the seed. This is all done in the below lines of code.

```
        # 1. PARSE INPUT (Convert mm/deg to m/rad)
        X = msg.data[0] / 1000.0 #convert to meters
        Y = msg.data[1] / 1000.0
        Z = msg.data[2] / 1000.0
        Roll = math.radians(msg.data[3]) #convert to radians
        Pitch = math.radians(msg.data[4])
        Yaw = math.radians(msg.data[5])

        # 2. SET TARGETS
        self.target_position = np.array([X, Y, Z])
        self.target_rotation = self.get_rotation_matrix(Roll, Pitch, Yaw)

        # 3. PREPARE SEED (Start from last known position)
        x0 = [self.current_full_joints[i] for i in self.motor_indices]
```

Now the optimization is ready to be run. This is done using scipy's built in minimize function. The line of code used for this is below.

```
        result = minimize(self.objective_function, x0, method='L-BFGS-B',
    bounds=self.joint_limits)
```

Lastly, the results must be interpreted. To do this, first it is checked if the minimization found a successful result. If it did not, the node logs an error. If it does, the solved joint angles are properly formatted into a list, and that list is formatted into a ROS2 message type to be published to the appropriate topic. This code is shown below.

```
        if result.success == True:
            for i, angle in enumerate(result.x):
                self.current_full_joints[self.motor_indices[i]] = angle

            msg = Float64MultiArray()
            msg.data = [float(a) for a in result.x]
            # DEBUG: See what we are sending
            self.get_logger().info(f"IK Success! Publishing angles (rad):
    {msg.data}")
```

```
            self.joint_pub.publish(msg)
        else:
            self.get_logger().info("Error: Check violations on units,
reachable space, etc.")
```

4.2 Cartesian Sender Node

Another node is needed to implement the inverse kinematics, that being the cartesian sender node. This node serves as a reliable source of target commands for the inverse kinematics solver node. The full script for this node can be found in **Appendix section 7.1.5**.

4.2.1 Writing the Cartesian Sender Node

The first step to this node is to define the publisher for the topic "/mirobot/command_cartesian", and to set up a test sequence of safe movements to test the various axes of motion on the mirobot. The code for this is shown below.

```
        self.cartesian_pub = self.create_publisher(Float64MultiArray, "/
mirobot/command_cartesian", 10)

        # TASK 2: DEFINE YOUR SEQUENCE
        # Format: [X(mm), Y(mm), Z(mm), Roll(deg), Pitch(deg), Yaw(deg),
DURATION(sec)]
        # Note: The 7th argument is how long the robot waits before the
NEXT move.
        self.coord_sequence = [
            # 1. Neutral Center
            [198.6, 0.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            # 2. X-Axis
            [220.0, 0.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            [180.0, 0.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            # 3. Y-Axis
            [198.6, 30.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            [198.6, -30.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            # 4. Z-Axis
            [198.6, 0.0, 210.0, 0.0, 0.0, 0.0, 2.0],
            [198.6, 0.0, 150.0, 0.0, 0.0, 0.0, 2.0],
        ]
```

The final number in each of the position sequences is the time given for the robot to complete the motion before moving on to the next position. The next portion of this node is to extract the position information from the defined test sequence and to publish it to the topic for controlling the robot. This is accomplished using the below code.

```
        current_step = self.coord_sequence[self.index]
        coords = current_step[0:6]
        duration = current_step[6]

        # TASK 4: PUBLISH
```

```
        # Create Float64MultiArray, assign coords to .data, and publish

        self.get_logger().info(
            f"Step {self.index}: Target {coords}. Waiting {duration}s"
        )
        self.index += 1

        msg = Float64MultiArray()
        msg.data = coords
        self.cartesian_pub.publish(msg)
```

The final step of this node is to set up the timer to track the duration between sending the next position command. A timer is created after each position command is sent using the below code.

```
        self.timer = self.create_timer(duration, self.execute_sequence)
```

4.3 Results

Step 4 was fully successful, properly utilizing a gradient descent optimization loop to solve inverse kinematics and then send the solved joint angles to the mirobot driver to control the physical robot. In **Figure 3** below, the RViz visualization and the physical robot position are pictured from the inverse kinematics being run.

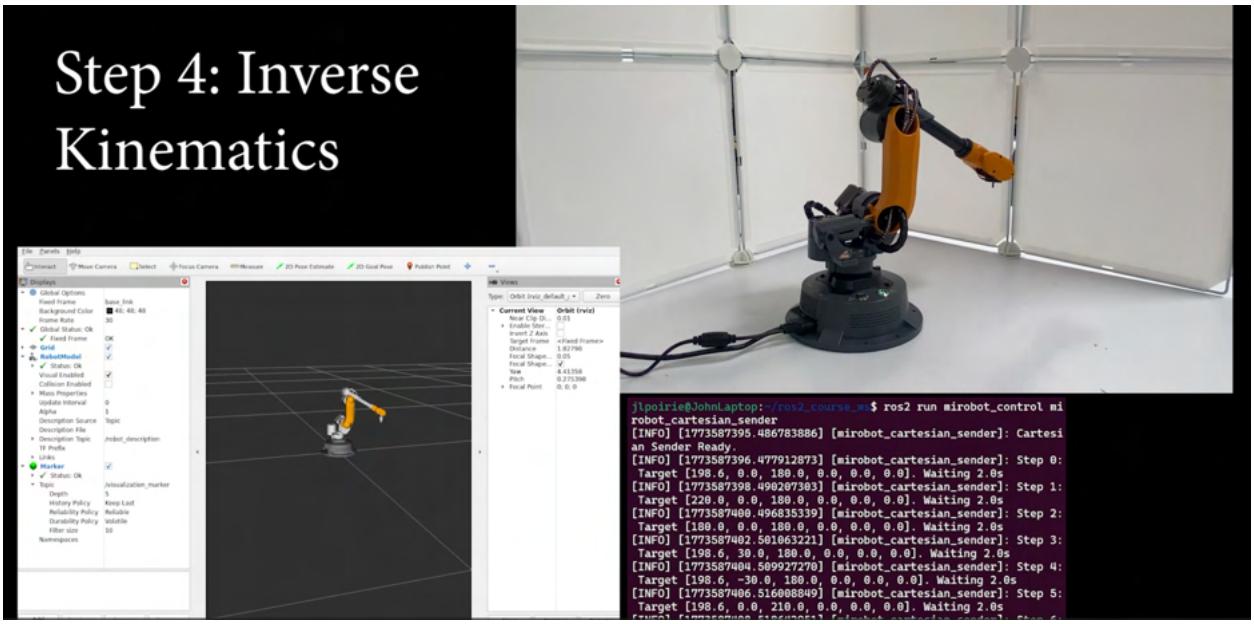


Figure 3: RViz2 and Physical Robot Results for Step 4.

4.4 Difficulties and Workarounds

The development of the inverse kinematics solver node and the cartesian sender node did not have any major complications, and the inverse kinematics solver was tested and validated to be working. The project document suggested using a heuristic weighted error of the form $(PositionError + \alpha) * RotationError$. However, this could cause issues with the optimization. If this error calculation was used, it could be possible that the rotation error could be nearly zero, while the position error is huge, and the objective function would find that the error is very small since the position error is being additively weighted, then multiplied with the rotation error. Instead, an additive objective error was used, of the form $(PositionError * \alpha) + (RotationError * \beta)$. In this case, a position weight α of 100 and a rotation weight β of 2 were used.

5 Step 5: Pure Pursuit

5.1 Node Architecture

The final section of this project is to implement pure pursuit. The complete script for the `mirobot_pure_pursuit.py` node is found in **Appendix section 7.1.6**.

5.1.1 Communication

To begin, topic names were matched from the provided `mirobot_draw.py`, then subscribers were created for the path states and joint states, as well as publishers for cartesian commands and RViz2 objects.

```
        # TODO: Create a Subscriber for the Path (Float64MultiArray)
        # self.sub_path = ...
        self.sub_path = self.create_subscription(
            Float64MultiArray, "/mirobot/target_path", self.path_callback,
    10)

        # TODO: Create a Subscriber for Joint States (JointState)
        # self.sub_joints = ...
        self.sub_joints = self.create_subscription(
            JointState, "/joint_states", self.joint_callback, 10)

        # TODO: Create a Publisher for Cartesian Commands (
   Float64MultiArray)
        # self.pub_cartesian = ...
        self.pub_cartesian = self.create_publisher(
            Float64MultiArray, "/mirobot/command_cartesian", 10)

        # TODO: Create a Publisher for RViz Markers (Marker)
        # self.pub_marker = ...
        self.pub_marker = self.create_publisher(
            Marker, "/visualization_marker", 10)
```

5.1.2 Robot Model

The URDF file was loaded from `mirobot_description` using `get_package_share_directory` and was used to build an ikpy chain of the robot links and joints. The forward kinematics are computed by adding a fixed base joint at 0.0 with `fill_joints` to the 6 joint angles, then getting XYZ out of the 4x4 homogeneous transformation matrix that `forward_kinematics` returns with the rotational matrix of the end effector's orientation as well as its position to update `current_pos`. So, `current_pos` updates in real time as the robot moves.

```
        try:
            pkg_share = get_package_share_directory("mirobot_description")
            urdf_file = os.path.join(pkg_share, "urdf", "
    mirobot_description.urdf")
            self.chain = ikpy.chain.Chain.from_urdf_file(urdf_file)
        except Exception as e:
            self.get_logger().warn(f"Could not load URDF: {e}")
            self.chain = None

        self.current_pos = [0.2, 0.0, 0.2]
```

```
        def joint_callback(self, msg):
            """Updates self.current_pos using Forward Kinematics."""

            # CALCAULATE FORWARD KINEMATICS AND UPDATE self.current_pos
            if self.chain is None or len(msg.position) < 6:
                return
            full_joints = [0.0] + list(msg.position[:6])
            if len(full_joints) < len(self.chain.links):
                full_joints += [0.0] * (len(self.chain.links) - len(
    full_joints))
            matrix = self.chain.forward_kinematics(full_joints)
            self.current_pos = list(matrix[:3, 3])
```

5.2 Phase 1

5.2.1 Get Start Point

Index 0 of the path is received from the user and set as the target the robot must reach before pure pursuit begins. So, the start point is available for distance checking and publishing.

```
        # TODO: Get the start point (index 0)
                start_point = self.path[0]
```

5.2.2 Publish Target Marker and Cartesian Command

The start point is published as both a red sphere in RViz2 and a cartesian command to the IK solver, so the robot physically moves toward the start of the path. The robot begins moving toward the start point and the target is visible in RViz2.

```
        # TODO: Publish the target marker and Cartesian Command to move to
    start point
                self.publish_target_marker(start_point)
                self.publish_cartesian(start_point)
```

5.2.3 Check Distance

The Euclidean distance between `current_pos` and the start point is calculated. If it is within the threshold, the algorithm continues to Phase 2. Occasionally, there would be an issue where, because the robot uses dead reckoning and not encoders, it has no way to read the actual joint angles without commands. This means that at startup it would publish zeros as place holders, but this is not the actual home position it is in at startup after homing the robot. To work around this issue, the approach threshold was raised from 0.020m to 0.200m so that Phase 2 can be implemented despite the current position being inaccurate. The tradeoff is that by essentially bypassing phase 1, we risk the robot jumping to a waypoint in the middle of the path if the robot is very far from the start point. So, the node correctly transitions from Phase 1 to Phase 2 once the robot is sufficiently close to the path start.

```
        # TODO: Check Distance
            d = self.dist(current_xy, start_point)
            # If distance < 0.020 (20mm):
            #    Set self.is_approaching = False
            #    Log "Target Reached"
            if d < 0.200: # changed from 0.05 - Sarah
                self.is_approaching = False
                self.get_logger().info("Start point reached. Pure Pursuit
    begins!")
            # Else:
            #    Print debug info occasionally
            else:
                self.tick_count += 1
                if self.tick_count % 20 == 0:
                    self.get_logger().info(f"Approaching start... dist={d
    *1000:.1f} mm")
```

5.3 Phase 2

Pure pursuit is now implemented into the loop. The speed logic skips ticks using `ticks_to_wait = max(1, 6 - speed_val)` so higher GUI speed values reduce wait time. The closest point search scans 30 points ahead of the last waypoint the robot was targeting - `last_idx`. The look-ahead target is found by stepping forward from the closest point until distance exceeds `self.lookahead (5cm)`. The max jump of 10 indices prevents large skips per tick. The lag check prevents `last_idx` from advancing if the robot hasn't caught up, and the path ends when the robot is within 50mm of the final point. So, the Robot follows the drawn path with controllable speed, look-ahead targeting, and path completion detection.

```
        # 1. SPEED LOGIC (Skip ticks based on speed_val)
```

```
            ticks_to_wait = max(1, 6 - self.speed_val)
            self.tick_count += 1
            if self.tick_count % ticks_to_wait != 0:
                # Keep holding current position
                target = self.path[self.last_idx]
                # Publish command...
                self.publish_target_marker(target)
                self.publish_cartesian(target)
                return

            # 2. FIND CLOSEST POINT (Search Logic)
            # Loop through path starting from self.last_idx to find
    minimum distance
            search_window = min(self.last_idx + 30, len(self.path))
            closest_idx = self.last_idx
            closest_dist = self.dist(current_xy, self.path[self.last_idx])

            for i in range(self.last_idx, search_window):
                d = self.dist(current_xy, self.path[i])
                if d < closest_dist:
                    closest_dist = d
                    closest_idx = i
            # 3. FIND LOOK-AHEAD TARGET
            # Loop forward from closest index until distance > self.
    lookahead
            target_idx = closest_idx
            for i in range(closest_idx, len(self.path)):
                if self.dist(current_xy, self.path[i]) >= self.lookahead:
                    target_idx = i
                    break
            else:
                target_idx = len(self.path) - 1
            # 4. UPDATE LAST INDEX (With checks)
            # Ensure we don't jump too far (speed governor)
            # Ensure we don't advance if lagging too much (lag check)
            max_jump = 10
            target_idx = min(target_idx, self.last_idx + max_jump)

            lag = self.dist(current_xy, self.path[self.last_idx])
            if lag < self.max_lag:
                self.last_idx = target_idx

            target = self.path[self.last_idx]

            # 5. PUBLISH COMMAND
            # Publish Marker
            # Publish Float64MultiArray to command_cartesian
            self.publish_target_marker(target)
            self.publish_cartesian(target)

            # 6. END CONDITION
            # If at end of list AND close to last point -> Stop
            if self.last_idx >= len(self.path) - 1:
                if self.dist(current_xy, self.path[-1]) < 0.050:
```

```
                    self.is_active = False
                    self.get_logger().info("Path complete!")
```

5.4 Launch File Update

The `mirobot_draw and mirobot_pure_pursuit` nodes were added to `robot_description.launch.py` so the draw GUI and pure pursuit tracker initializes with the package launch.

5.5 Results

The results from this step were ultimately successful, though had various difficulties throughout. The pure pursuit algorithm was successfully implemented, along with the GUI that allows drawing of the paths. This also properly connected with RViz, showing the path in RViz when drawn and sent. **Figure** 4 below shows the RViz visualization and the physical robot positioning, as well as the pure pursuit interface.

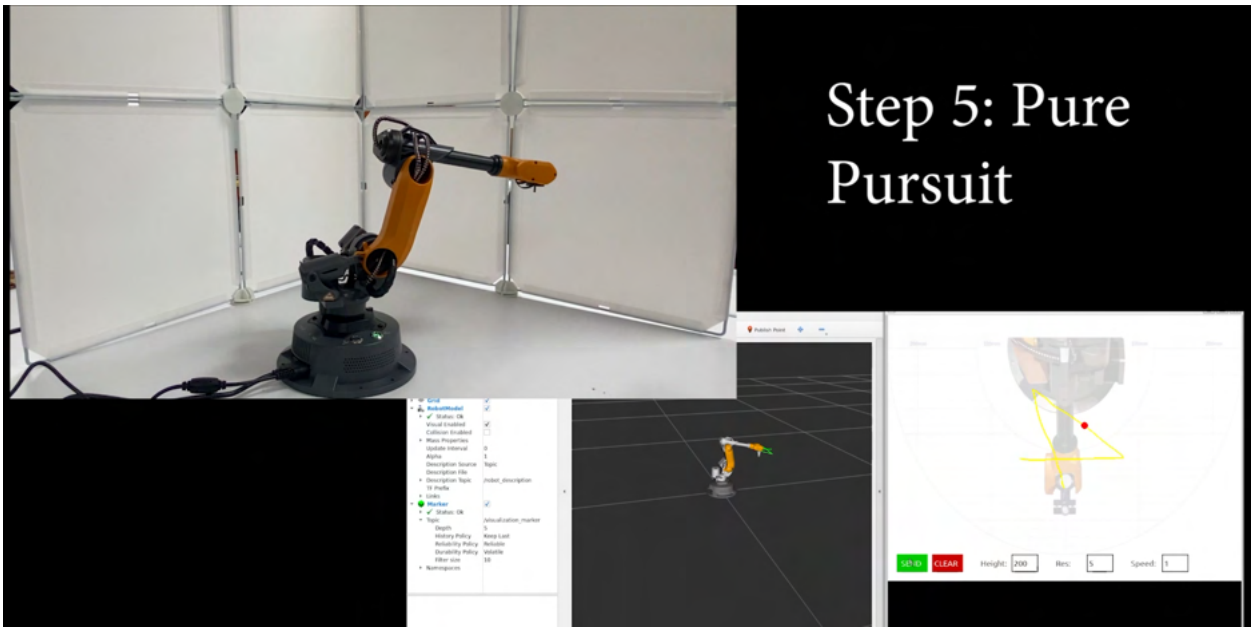


Figure 4: RViz2 and Physical Robot Results for Step 5.

5.6 Difficulties and Workarounds

5.6.1 Cartesian Command Publisher

A helper function was added to convert the look-ahead target from meters to mm and publish to the IK solver, with correctly formatted commands for the IK solver. Note: The inclusion of this helper function was a recommendation from Claude AI.

```
        # helper function to publish target postion in correct units to IK
    solver
        def publish_cartesian(self, target_xy):
            msg = Float64MultiArray()
            msg.data = [
                target_xy[0] * 1000.0,  # X in mm
                target_xy[1] * 1000.0,  # Y in mm
                self.z_height,          # Z in mm
                0.0,                    # Roll
                90.0,                   # Pitch (tool pointing down)
                0.0,                    # Yaw
                ]
            self.pub_cartesian.publish(msg)
```

5.6.2 Look-Ahead Distance

The look-ahead distance was increased from 0.025 to 0.05 during testing. This helped improve the sensitivity and jitters of the robot's movements by not fixating on every small change in the path caused by the user's imperfect drawing. However, there is a tradeoff: the robot sometimes cuts corners if the corners are too sharp. This ultimately comes down to a user preference for how important each of these behaviors are, comparatively.

```
        # Path State
            self.path = []
            self.is_active = False
            self.is_approaching = False
            self.last_idx = 0
            self.tick_count = 0

            # Tuning
            self.lookahead = 0.05 # increased from 0.025 -Sarah
            self.max_lag = 0.050
```

6 Conclusion

Overall, this project was very successful. The provided files gave great guidance on where to continue developing the project, and the project document was very well put together and defined. No major issues came up during development that prevented the completion of this project. The majority of issues came from ROS2 quirks, with strict topic naming and formatting structures. The success of the inverse kinematics solver along with the pure pursuit implementation proved to be an incredibly valuable real-world robotics implementation. This project gave significant insight into real robotics, with physical limitations, and was crucial in the development of ROS skills.

7 Appendix

7.1 Code

7.1.1 Robot Description Launch File

```
#!/usr/bin/env python3

# imports
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    description_pkg_name = "mirobot_description"
    control_pkg_name = "mirobot_control"

    # Load URDF file
    urdf_file_name = 'mirobot_description.urdf'
    urdf = os.path.join(get_package_share_directory(description_pkg_name),
    'urdf', urdf_file_name)

    with open(urdf, 'r') as infp:
        robot_desc = infp.read()

    #RViz config path
    rviz_config_file = os.path.join(get_package_share_directory(
   description_pkg_name), 'rviz', 'mirobot.rviz')

    return LaunchDescription([
        # robot state publisher node
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            parameters=[{'robot_description': robot_desc}]
        ),

        # rviz2 node (use ./rviz/mirobot.rviz configuration file)
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
            output='screen',
            arguments=['-d', rviz_config_file]
        ),

        # mirobot driver node
        Node(
            package='mirobot_control',
            executable='mirobot_driver',
```

```
            name='mirobot_driver',
            output='screen',
        ),

        # inverse kinematics solver node
        Node(
            package='mirobot_control',
            executable='mirobot_ik_solver',
            name='mirobot_ik_solver',
            output='screen',
        ),

        # mirobot draw node
        Node(
            package='mirobot_control',
            executable='mirobot_draw',
            name='mirobot_draw',
            output='screen',
        ),

        # mirobot pure pursuit node
        Node(
            package='mirobot_control',
            executable='mirobot_pure_pursuit',
            name='mirobot_pure_pursuit',
            output='screen',
        ),

    ])
```

7.1.2 Mirobot Commander Node

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray
import time



class MirobotSender(Node):
    def __init__(self):
        super().__init__("mirobot_sender")

        # WRITE YOUR PUBLISHER HERE TO PUBLISH JOINTS
        self.joint_pub = self.create_publisher(Float64MultiArray, "/
   mirobot/command_joints", 10)

        self.get_logger().info("Sender Node Ready.")

    def send_joints(self, joints):
        """
        joints: list of 6 floats (radians)
        """
```

```
        # WRITE YOUR PUBLISHING CODE HERE TO PUBLISH JOINTS
        msg = Float64MultiArray()
        msg.data = joints
        self.joint_pub.publish(msg)
        pass


def main(args=None):
    rclpy.init(args=args)
    sender = MirobotSender()

    # [J1, J2, J3, J4, J5, J6]
    home_pose = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    pose_A = [0.5, -0.2, 0.2, 0.0, 0.0, 0.0]
    pose_B = [-0.5, 0.3, 0.5, 0.0, 0.0, 0.0]

    try:
        sender.send_joints(home_pose)
        time.sleep(2) #generally bad practice, but this node never listens
    for anything, so it's fine
        sender.send_joints(pose_A)
        time.sleep(2)
        sender.send_joints(pose_B)
        time.sleep(2)
    except KeyboardInterrupt:
        pass
    finally:
        sender.destroy_node()
        rclpy.shutdown()
        pass


if __name__ == "__main__":
    main()
```

7.1.3 GUI Bridge Node

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
import math
from ament_index_python.packages import get_package_share_directory
import os
import ikpy.chain
from std_msgs.msg import Float64MultiArray
from sensor_msgs.msg import JointState


class GuiBridge(Node):
    def __init__(self):
        super().__init__("gui_bridge")

        # TASK 1: LOAD ROBOT MODEL (URDF)
        # Use get_package_share_directory to find "mirobot_description"
```

```
        urdf_file_name = 'mirobot_description.urdf'
        urdf = os.path.join(get_package_share_directory('
mirobot_description'), 'urdf', urdf_file_name)
        # Load the chain using ikpy.chain.Chain.from_urdf_file()
        self.chain = ikpy.chain.Chain.from_urdf_file(urdf)

        # TASK 2: SETUP COMMUNICATION
        # Create a Subscriber to Joint States from GUI (NOTE: You should
subscribe to a different topic than the real robot to avoid conflicts,
you can remap the topic when running joint state publisher gui)
        self.subscription = self.create_subscription(JointState, "/gui/
joint_state", self.listener_callback, 10)

        # Create a Publisher to Robot Driver
        self.joint_pub = self.create_publisher(Float64MultiArray, "/
mirobot/command_joints", 10)

        self.get_logger().info("GUI Bridge with FK Feedback Started.")

    def rotation_matrix_to_euler(self, R):
        """
        Helper: Converts a 3x3 Rotation Matrix to Euler Angles (Roll,
Pitch, Yaw).
        """
        sy = math.sqrt(R[0, 0] * R[0, 0] + R[1, 0] * R[1, 0])
        singular = sy < 1e-6

        if not singular:
            x = math.atan2(R[2, 1], R[2, 2])
            y = math.atan2(-R[2, 0], sy)
            z = math.atan2(R[1, 0], R[0, 0])
        else:
            x = math.atan2(-R[1, 2], R[1, 1])
            y = math.atan2(-R[2, 0], sy)
            z = 0

        return [math.degrees(x), math.degrees(y), math.degrees(z)]

    def listener_callback(self, msg):
        # TASK 3: BRIDGE THE MESSAGE
        # The GUI sends a JointState message (msg.position).
        # Create the new message, populate .data, and publish it.
        js_msg = Float64MultiArray()
        js_msg.data = msg.position
        self.joint_pub.publish(js_msg)


        # TASK 4: CALCULATE FORWARD KINEMATICS
        # Use self.chain.forward_kinematics() to get the 4x4
transformation matrix.
        # Note: You might need to prepend 0.0 to the joints list to
account for the base link.
        transf_mat = self.chain.forward_kinematics([0.0] + list(msg.
position))
```

```
        # TASK 5: EXTRACT & PRINT
        # Extract X, Y, Z from the matrix (Index [0,3], [1,3], [2,3])
        X = float(transf_mat[0,3])
        Y = float(transf_mat[1,3])
        Z = float(transf_mat[2,3])
        # Use the helper function above to get rotation.
        rotations = self.rotation_matrix_to_euler(transf_mat[0:3, 0:3]) #
   list, [roll, pitch, yaw]
        # Log the result to the console.
        self.get_logger().info(f"Positions = [{X}, {Y}, {Z}]")
        self.get_logger().info(f"Rotations = {rotations}")


def main(args=None):
    rclpy.init(args=args)
    node = GuiBridge()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == "__main__":
    main()
```

7.1.4 Inverse Kinematics Solver Node

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
import numpy as np
import math
from ament_index_python.packages import get_package_share_directory
import os
import ikpy.chain
from std_msgs.msg import Float64MultiArray
from scipy.optimize import minimize

class MirobotIKSolver(Node):
    def __init__(self):
        super().__init__("mirobot_ik_solver")

        # 1. SUBSCRIBE TO CARTESIAN TARGETS (POSITION + ORIENTATION)
        self.subscription = self.create_subscription(Float64MultiArray, "/
   mirobot/command_cartesian", self.cartesian_callback, 10)

        # 2. PUBLISH TO ROBOT DRIVER
        self.joint_pub = self.create_publisher(Float64MultiArray, "/
   mirobot/command_joints", 10)
```

```
        # 3. LOAD ROBOT MODEL
        urdf_file_name = 'mirobot_description.urdf'
        urdf = os.path.join(get_package_share_directory('
    mirobot_description'), 'urdf', urdf_file_name)

        # We have 7 links total (Base + 6 Motors).
        # We set index 0 (Base) to False, and indices 1-6 (Motors) to True
    .
        self.chain = ikpy.chain.Chain.from_urdf_file(
            urdf,
            active_links_mask=[False, True, True, True, True, True, True]
        )

        # 4. DEFINE MOTOR INDICES (Joints 1-6)
        self.motor_indices = [1, 2, 3, 4, 5, 6]

        # 5. EXTRACT JOINT LIMITS (BOUNDS)
        self.joint_limits = []
        for i in self.motor_indices:
            self.joint_limits.append(self.chain.links[i].bounds)

        # Seed: Where the robot starts looking for a solution (Start at "
    all zeros")
        self.current_full_joints = np.zeros(len(self.chain.links))

        # Give the shoulder and elbow a small "human-like" bend (in
    radians)
        # This helps prevent singularities on the very first move.
        self.current_full_joints[2] = 0.1  # Joint 2
        self.current_full_joints[3] = 0.2  # Joint 3

        self.get_logger().info("Gradient IK Solver Ready.")

    def get_rotation_matrix(self, r, p, y):
        """Helper to convert Euler angles to a Rotation Matrix."""
        cx, sx = math.cos(r), math.sin(r)
        cy, sy = math.cos(p), math.sin(p)
        cz, sz = math.cos(y), math.sin(y)
        Rx = np.array([[1, 0, 0], [0, cx, -sx], [0, sx, cx]])
        Ry = np.array([[cy, 0, sy], [0, 1, 0], [-sy, 0, cy]])
        Rz = np.array([[cz, -sz, 0], [sz, cz, 0], [0, 0, 1]])
        return Rz @ Ry @ Rx

    def objective_function(self, active_joints):
        """
        The 'Cost Function' for the optimizer.
        Goal: Return a low number if the robot is close to target, high if
     far away.
        """
        # TASK 1: MAP OPTIMIZER INPUT TO FULL ROBOT CHAIN
        # 'active_joints' only contains the 6 motor angles.
        # Create a list 'full_joints' that matches len(self.chain.links).
        # Fill the indices specified in self.motor_indices with values
    from active_joints.
```

```
        full_joints = self.current_full_joints.copy() #avoid overwirting
the original
        for i, angle in enumerate(active_joints):
            full_joints[self.motor_indices[i]] = angle

        # TASK 2: CALCULATE FORWARD KINEMATICS
        # Use the chain object to find where the robot IS based on these
angles.
        # Extract the current Position (XYZ) and Rotation (Matrix).
        transf_mat = self.chain.forward_kinematics(full_joints)

        X = float(transf_mat[0,3])
        Y = float(transf_mat[1,3])
        Z = float(transf_mat[2,3])
        # Use the helper function above to get rotation.
        cur_rotation = transf_mat[0:3, 0:3] #list, [roll, pitch, yaw]

        # TASK 3: CALCULATE ERROR
        # Calculate the distance between the 'current' position and the '
target' position.
        # Calculate the difference between the 'current' rotation and '
target' rotation.
        pos_error = np.linalg.norm(self.target_position - transf_mat[0:3,
3])
        rot_error = np.linalg.norm(self.target_rotation - cur_rotation)

        # TASK 4: RETURN WEIGHTED COST
        # Return the total error. (Hint: Position error is usually
weighted higher than rotation).
        return (pos_error * 100.0) + (rot_error * 2.0)

    def cartesian_callback(self, msg):
        if len(msg.data) != 6:
            return

        # 1. PARSE INPUT (Convert mm/deg to m/rad)
        X = msg.data[0] / 1000.0 #convert to meters
        Y = msg.data[1] / 1000.0
        Z = msg.data[2] / 1000.0
        Roll = math.radians(msg.data[3]) #convert to radians
        Pitch = math.radians(msg.data[4])
        Yaw = math.radians(msg.data[5])

        # 2. SET TARGETS
        self.target_position = np.array([X, Y, Z])
        self.target_rotation = self.get_rotation_matrix(Roll, Pitch, Yaw)

        # 3. PREPARE SEED (Start from last known position)
        x0 = [self.current_full_joints[i] for i in self.motor_indices]


        # TASK 5: RUN OPTIMIZATION
        # Use scipy.optimize.minimize().
```

```
        # You need to pass the objective function, the seed (x0), the
    method ('L-BFGS-B'), and the bounds.
        result = minimize(self.objective_function, x0, method='L-BFGS-B',
    bounds=self.joint_limits)

        # TASK 6: PROCESS RESULT
        # If the optimization was successful (result.success):
        #   1. Update self.current_full_joints with the new solution.
        #   2. Publish the result to "/mirobot/command_joints".
        # Else:
        #   Log an error.

        if result.success == True:
            for i, angle in enumerate(result.x):
                self.current_full_joints[self.motor_indices[i]] = angle

            msg = Float64MultiArray()
            msg.data = [float(a) for a in result.x]
            # DEBUG: See what we are sending
            self.get_logger().info(f"IK Success! Publishing angles (rad):
    {msg.data}")

            self.joint_pub.publish(msg)
        else:
            self.get_logger().info("Error: Check violations on units,
    reachable space, etc.")


def main(args=None):
    rclpy.init(args=args)
    node = MirobotIKSolver()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

7.1.5 Cartesian Sender Node

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray

class MirobotCartesianSender(Node):
    def __init__(self):
        super().__init__("mirobot_cartesian_sender")

        # TASK 1: CREATE PUBLISHER
        # Publish to topic: "/mirobot/command_cartesian"
        # Message type: Float64MultiArray
        self.cartesian_pub = self.create_publisher(Float64MultiArray, "/
```

```
mirobot/command_cartesian", 10)

        # TASK 2: DEFINE YOUR SEQUENCE
        # Format: [X(mm), Y(mm), Z(mm), Roll(deg), Pitch(deg), Yaw(deg),
DURATION(sec)]
        # Note: The 7th argument is how long the robot waits before the
NEXT move.
        self.coord_sequence = [
            # 1. Neutral Center
            [198.6, 0.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            # 2. X-Axis
            [220.0, 0.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            [180.0, 0.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            # 3. Y-Axis
            [198.6, 30.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            [198.6, -30.0, 180.0, 0.0, 0.0, 0.0, 2.0],
            # 4. Z-Axis
            [198.6, 0.0, 210.0, 0.0, 0.0, 0.0, 2.0],
            [198.6, 0.0, 150.0, 0.0, 0.0, 0.0, 2.0],
        ]

        self.index = 0

        # Start the first timer (waits 1.0s before starting the sequence)
        self.timer = self.create_timer(1.0, self.execute_sequence)
        self.get_logger().info("Cartesian Sender Ready.")

    def execute_sequence(self):
        # 1. STOP PREVIOUS TIMER
        # We must cancel the old timer so it doesn't keep firing at the
old rate.
        self.timer.cancel()

        # Check if we have reached the end of the list
        if self.index >= len(self.coord_sequence):
            self.get_logger().info("Sequence Complete.")
            return

        # TASK 3: EXTRACT DATA
        # Get the current row from self.coord_sequence
        # Separate it into 'coords' (first 6 items) and 'duration' (7th
item)
        current_step = self.coord_sequence[self.index]
        coords = current_step[0:6]
        duration = current_step[6]

        # TASK 4: PUBLISH
        # Create Float64MultiArray, assign coords to .data, and publish

        self.get_logger().info(
            f"Step {self.index}: Target {coords}. Waiting {duration}s"
        )
        self.index += 1
```

```
        msg = Float64MultiArray()
        msg.data = coords
        self.cartesian_pub.publish(msg)

        # TASK 5: SET DYNAMIC TIMER
        # Create a NEW timer that waits for 'duration' seconds before
   calling
        # self.execute_sequence again.
        self.timer = self.create_timer(duration, self.execute_sequence)


def main(args=None):
    rclpy.init(args=args)
    node = MirobotCartesianSender()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

7.1.6 Mirobot Pure Pursuit Node

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from rclpy.parameter import Parameter
from rcl_interfaces.msg import SetParametersResult
from visualization_msgs.msg import Marker
from geometry_msgs.msg import Point
import math
from std_msgs.msg import Float64MultiArray
from sensor_msgs.msg import JointState
import ikpy.chain
import numpy as np
import os
from ament_index_python.packages import get_package_share_directory


class MirobotPurePursuit(Node):
    def __init__(self):
        super().__init__("mirobot_pure_pursuit")

        # 1. PARAMETERS (Z-Height and Speed)
        self.declare_parameter("z_height", 200.0)
        self.declare_parameter("speed", 1)
        self.add_on_set_parameters_callback(self.param_callback)

        self.z_height = self.get_parameter("z_height").value
        self.speed_val = self.get_parameter("speed").value

        # 2. COMMUNICATION (TODO: Complete this section)
        # READ mirobot_draw.py to find the correct Topic Names!
```

```
        # TODO: Create a Subscriber for the Path (Float64MultiArray)
        # self.sub_path = ...
        self.sub_path = self.create_subscription(
            Float64MultiArray, "/mirobot/target_path", self.path_callback,
 10)

        # TODO: Create a Subscriber for Joint States (JointState)
        # self.sub_joints = ...
        self.sub_joints = self.create_subscription(
            JointState, "/joint_states", self.joint_callback, 10)

        # TODO: Create a Publisher for Cartesian Commands (
Float64MultiArray)
        # self.pub_cartesian = ...
        self.pub_cartesian = self.create_publisher(
            Float64MultiArray, "/mirobot/command_cartesian", 10)

        # TODO: Create a Publisher for RViz Markers (Marker)
        # self.pub_marker = ...
        self.pub_marker = self.create_publisher(
            Marker, "/visualization_marker", 10)

        # 3. ROBOT MODEL (IKPy)
        try:
            pkg_share = get_package_share_directory("mirobot_description")
            urdf_file = os.path.join(pkg_share, "urdf", "
mirobot_description.urdf")
            self.chain = ikpy.chain.Chain.from_urdf_file(urdf_file)
        except Exception as e:
            self.get_logger().warn(f"Could not load URDF: {e}")
            self.chain = None

        self.current_pos = [0.2, 0.0, 0.2]

        # Path State
        self.path = []
        self.is_active = False
        self.is_approaching = False
        self.last_idx = 0
        self.tick_count = 0

        # Tuning
        self.lookahead = 0.025
        self.max_lag = 0.050

        # Run loop at 20Hz (0.05s)
        self.timer = self.create_timer(0.05, self.control_loop)

    def param_callback(self, params):
        """Updates z_height and speed when changed via GUI."""
        for param in params:
            if param.name == "z_height":
                self.z_height = float(param.value)
```

```
                self.get_logger().info(f"Z-Height Updated: {self.z_height}
")
            elif param.name == "speed":
                if param.type_ == Parameter.Type.INTEGER:
                    self.speed_val = int(param.value)
                    self.get_logger().info(f"Speed Updated: {self.
speed_val}")
        return SetParametersResult(successful=True)

    def joint_callback(self, msg):
        """Updates self.current_pos using Forward Kinematics."""

        # CALCAULATE FORWARD KINEMATICS AND UPDATE self.current_pos
        if self.chain is None or len(msg.position) < 6:
            return
        full_joints = [0.0] + list(msg.position[:6])
        if len(full_joints) < len(self.chain.links):
            full_joints += [0.0] * (len(self.chain.links) - len(
full_joints))
        matrix = self.chain.forward_kinematics(full_joints)
        self.current_pos = list(matrix[:3, 3])

    def path_callback(self, msg):
        """Receives the path from the GUI."""
        data = msg.data
        new_path = []
        # Convert flat list [x1, y1, x2, y2...] to [(x1,y1), (x2,y2)...]
        for i in range(0, len(data), 2):
            new_path.append((data[i], data[i + 1]))

        if len(new_path) > 2:
            self.path = new_path
            self.is_active = True

            # Reset State
            self.is_approaching = True
            self.last_idx = 0
            self.tick_count = 0

            self.publish_path_marker()
            self.get_logger().info("Path Received. Approaching Start...")

    def dist(self, p1, p2):
        return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)

    def publish_path_marker(self):
        """Publishes the entire path as a YELLOW LINE in RViz."""
        marker = Marker()
        marker.header.frame_id = "base_link"
        marker.header.stamp = self.get_clock().now().to_msg()
        marker.ns = "path"
        marker.id = 0
        marker.type = Marker.LINE_STRIP
        marker.action = Marker.ADD
```

```
        marker.scale.x = 0.003
        marker.color.a = 1.0
        marker.color.g = 1.0  # Yellow
        for p in self.path:
            pt = Point()
            pt.x, pt.y, pt.z = p[0], p[1], self.z_height / 1000.0
            marker.points.append(pt)
        self.pub_marker.publish(marker)

    def publish_target_marker(self, target):
        """Publishes the current target as a RED SPHERE in RViz."""
        marker = Marker()
        marker.header.frame_id = "base_link"
        marker.header.stamp = self.get_clock().now().to_msg()
        marker.ns = "target"
        marker.id = 1
        marker.type = Marker.SPHERE
        marker.action = Marker.ADD
        marker.pose.position.x = target[0]
        marker.pose.position.y = target[1]
        marker.pose.position.z = self.z_height / 1000.0
        marker.scale.x = 0.015
        marker.scale.y = 0.015
        marker.scale.z = 0.015
        marker.color.a = 1.0
        marker.color.r = 1.0  # Red
        self.pub_marker.publish(marker)

    # helper function to publish target postion in correct units to IK
    solver - Sarah
    def publish_cartesian(self, target_xy):
        msg = Float64MultiArray()
        msg.data = [
            target_xy[0] * 1000.0,  # X in mm
            target_xy[1] * 1000.0,  # Y in mm
            self.z_height,          # Z in mm
            0.0,                    # Roll
            0.0,                   # Pitch (90.0 would be tool pointing
    down)
            0.0,                    # Yaw
            ]
        self.pub_cartesian.publish(msg)

    def control_loop(self):
        """Main Pure Pursuit Loop."""
        if not self.is_active or not self.path:
            return

        current_xy = (self.current_pos[0], self.current_pos[1])

        # =========================================
        # PHASE 1: STRICT APPROACH (WAIT LOGIC)
        # =========================================
        if self.is_approaching:
```

```
            # TODO: Get the start point (index 0)
            start_point = self.path[0]

            # TODO: Publish the target marker and Cartesian Command to
move to start point
            self.publish_target_marker(start_point)
            self.publish_cartesian(start_point)

            # TODO: Check Distance
            d = self.dist(current_xy, start_point)
            # If distance < 0.020 (20mm):
            #    Set self.is_approaching = False
            #    Log "Target Reached"
            if d < 0.020: # changed from 0.05 - Sarah
                self.is_approaching = False
                self.get_logger().info("Start point reached. Pure Pursuit
begins!")
            # Else:
            #    Print debug info occasionally
            else:
                self.tick_count += 1
                if self.tick_count % 20 == 0:
                    self.get_logger().info(f"Approaching start... dist={d
*1000:.1f} mm")

            return  # STOP HERE. Do not run Phase 2 yet.

        # ==========================================
        # PHASE 2: PATH FOLLOWING
        # ==========================================

        # 1. SPEED LOGIC (Skip ticks based on speed_val)
        ticks_to_wait = max(1, 6 - self.speed_val)
        self.tick_count += 1
        if self.tick_count % ticks_to_wait != 0:
            # Keep holding current position
            target = self.path[self.last_idx]
            # Publish command...
            self.publish_target_marker(target)
            self.publish_cartesian(target)
            return

        # 2. FIND CLOSEST POINT (Search Logic)
        # Loop through path starting from self.last_idx to find minimum
distance
        search_window = min(self.last_idx + 10, len(self.path))
        closest_idx = self.last_idx
        closest_dist = self.dist(current_xy, self.path[self.last_idx])

        for i in range(self.last_idx, search_window):
            d = self.dist(current_xy, self.path[i])
            if d < closest_dist:
                closest_dist = d
                closest_idx = i
```

```
        # 3. FIND LOOK-AHEAD TARGET
        # Loop forward from closest index until distance > self.lookahead
        target_idx = closest_idx
        for i in range(closest_idx, len(self.path)):
            if self.dist(current_xy, self.path[i]) >= self.lookahead:
                target_idx = i
                break
        else:
            target_idx = len(self.path) - 1
        # 4. UPDATE LAST INDEX (With checks)
        # Ensure we don't jump too far (speed governor)
        # Ensure we don't advance if lagging too much (lag check)
        max_jump = 3
        target_idx = min(target_idx, self.last_idx + max_jump)

        lag = self.dist(current_xy, self.path[self.last_idx])
        if lag < self.max_lag:
            self.last_idx = target_idx

        target = self.path[self.last_idx]

        # 5. PUBLISH COMMAND
        # Publish Marker
        # Publish Float64MultiArray to command_cartesian
        self.publish_target_marker(target)
        self.publish_cartesian(target)

        # 6. END CONDITION
        # If at end of list AND close to last point -> Stop
        if self.last_idx >= len(self.path) - 1:
            if self.dist(current_xy, self.path[-1]) < 0.050:
                self.is_active = False
                self.get_logger().info("Path complete!")


def main(args=None):
    rclpy.init(args=args)
    node = MirobotPurePursuit()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```